

MediLink LK – AI-Enabled Smart Healthcare Appointment & Telemedicine Platform

SE3020 – Distributed Systems (Assignment 1) • Report • Date: 2026-04-18

Repository scope: Microservices-based healthcare platform demonstrating service discovery, API gateway with JWT/RBAC, per-service databases, async messaging, and end-to-end workflows (appointments → payments → telemedicine → notifications → prescriptions).

Deliverables: This report is designed to be exported as `report.pdf`. It contains no screenshots; code excerpts are included as plain text.

Table of Contents

1. Introduction
2. Architecture
3. Service Interfaces (API contracts)
4. Workflows
5. Security & Authentication
6. Deployment
7. Individual Contributions
8. Appendix – Code Written (text excerpts)

1. Introduction

MediLink LK is a distributed, microservices-based healthcare platform concept that supports doctor onboarding/verification, appointment booking, online payments, telemedicine session provisioning, consultation completion, patient notifications, and digital prescriptions.

Goals (assignment-aligned):

- Demonstrate microservices separation and independent deployment
- Use API Gateway + Service Discovery for routing
- Implement JWT authentication and role-based access control
- Use asynchronous messaging (RabbitMQ) for event-driven workflows
- Maintain per-service data ownership (PostgreSQL per service)
- Provide a repeatable end-to-end proof run (smoke test)

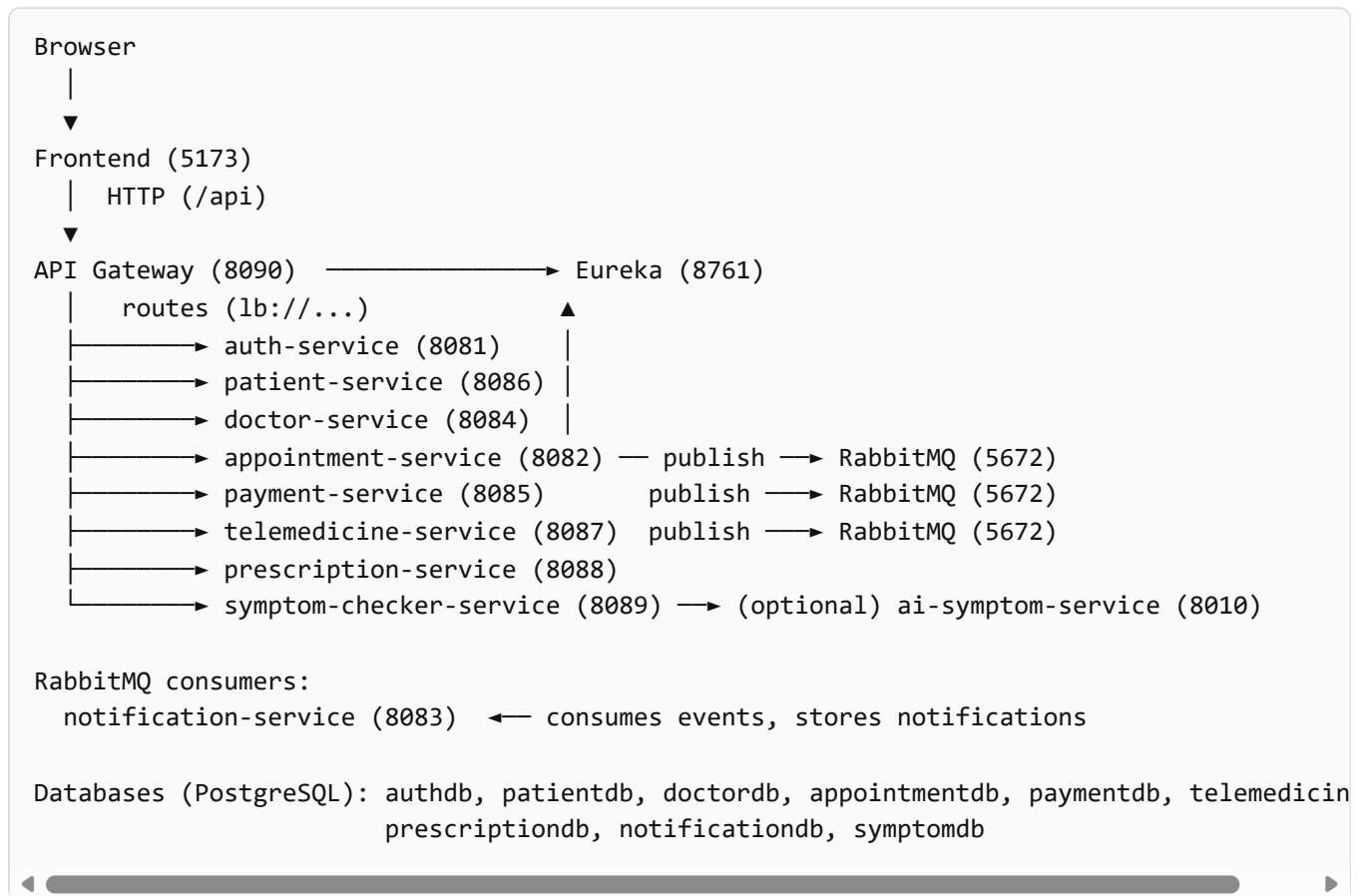
2. Architecture (High-level)

Key components:

- **Frontend** (React + Vite, served via Nginx)
- **API Gateway** (Spring Cloud Gateway) for routing + JWT validation + RBAC
- **Service Discovery** (Eureka) for dynamic registration
- **Domain services:** auth, patient, doctor, appointment, payment, telemedicine, notification, prescription, symptom checker

- **RabbitMQ** for asynchronous events
- **PostgreSQL** per service for data ownership

2.1 Architectural Diagram (text)



3. Service Interfaces (API contracts)

All endpoints are accessed through the API Gateway in normal operation. Identity is injected by the gateway into X-User-Id and X-User-Role headers.

3.1 Auth Service

- POST /api/auth/register – Register and receive JWT
- POST /api/auth/login – Login and receive JWT
- GET /api/admin/users – Admin list/search users
- DELETE /api/admin/users/{id} – Admin delete a non-admin user

3.2 Doctor Service

- POST /api/doctors/me/profile – Upsert doctor profile
- POST /api/doctors/me/profile-photo – Upload doctor profile photo
- GET /api/doctors/me/profile – Get own profile
- GET /api/doctors – Search verified doctors (optional specialization filter)
- GET /api/doctors/{doctorId}/availability – Get availability blocks
- GET /api/doctors/{doctorId}/availability/slots – Generate slots
- GET /api/doctors/{doctorId}/availability/validate – Validate a proposed slot time

- GET /api/admin/doctors/pending – Admin list pending verifications
- GET /api/admin/doctors/recent-decisions – Admin recent approve/reject decisions
- POST /api/admin/doctors/{doctorId}/approve – Admin approve doctor
- POST /api/admin/doctors/{doctorId}/reject – Admin reject doctor

3.3 Patient Service

- GET /api/patients/me/profile – Get own patient profile
- POST /api/patients/me/profile – Upsert patient profile
- POST /api/patients/me/profile-photo – Upload patient profile photo
- POST /api/patients/me/reports – Upload medical report
- GET /api/patients/me/reports – List reports
- GET /api/patients/me/reports/{id}/download – Download report
- DELETE /api/patients/me/reports/{id} – Delete report
- GET /api/admin/patients – Admin list patient profiles
- GET /api/admin/patients/{userId}/profile – Admin get a patient profile
- GET /api/admin/patients/{userId}/reports – Admin list patient reports
- GET /api/admin/patients/{userId}/reports/{id}/download – Admin download patient report

3.4 Appointment Service

- POST /api/appointments – Create appointment
- GET /api/appointments – List patient appointments
- GET /api/appointments/doctor/me – List doctor appointments
- PUT /api/appointments/doctor/me/{id}/approval – Doctor approve/reject an appointment
- GET /api/appointments/available-slots – Available slots for a doctor
- PUT /api/appointments/{id}/reschedule – Reschedule appointment
- DELETE /api/appointments/{id} – Cancel appointment
- GET /api/appointments/stream – Patient SSE stream
- GET /api/appointments/doctor/me/stream – Doctor SSE stream
- GET /api/appointments/admin/stream – Admin SSE stream
- GET /api/admin/appointments – Admin list appointments
- DELETE /api/admin/appointments/{id} – Admin cancel appointment

3.5 Payment Service

- POST /api/payments/intents/payhere – Create PayHere intent
- POST /api/payments/intents/stripe – Create Stripe checkout intent
- POST /api/payments/stripe/confirm – Confirm Stripe session
- POST /api/payments/callback/payhere – PayHere server-to-server notify
- POST /api/payments/callback/stripe – Stripe webhook
- GET /api/admin/payments – Admin list payments
- POST /api/admin/payments/{paymentId}/review – Admin review payment
- POST /api/admin/payments/{paymentId}/dispute – Admin flag dispute
- POST /api/admin/payments/{paymentId}/refund – Admin refund

3.6 Telemedicine Service

- GET /api/telemedicine/sessions/appointment/{appointmentId} – Get session info (join URL)
- POST /api/telemedicine/sessions/appointment/{appointmentId}/complete – Mark consultation completed

3.7 Notification Service

- GET /api/notifications – List current user notifications
- GET /api/admin/notifications – Admin list notifications

3.8 Prescription Service

- POST /api/prescriptions/doctor/me – Doctor issue prescription
- GET /api/prescriptions/doctor/me – Doctor list issued prescriptions
- GET /api/prescriptions/patient/me – Patient list prescriptions
- GET /api/prescriptions/{id} – Get prescription (user-scoped)
- GET /api/admin/prescriptions – Admin list prescriptions

3.9 Symptom Checker Service (optional enhancement)

- POST /api/symptoms/check – Create assessment and receive recommendations
- GET /api/symptoms/history – List user assessment history
- GET /api/symptoms/{id} – Get a specific assessment
- GET /api/admin/symptoms – Admin view recent assessments

4. Workflows

4.1 Doctor onboarding + admin verification

1. Doctor registers/logins via /api/auth/*.
2. Doctor creates profile via POST /api/doctors/me/profile.
3. Admin approves via POST /api/admin/doctors/{doctorId}/approve.

4.2 Appointment booking → payment → confirmation

1. Patient books appointment via POST /api/appointments (initially pending payment).
2. Doctor approves via PUT /api/appointments/doctor/me/{id}/approval.
3. Patient creates payment intent via POST /api/payments/intents/*.
4. Payment callback/webhook marks payment completed and publishes an event.
5. Appointment transitions to confirmed asynchronously (event-driven).

4.3 Telemedicine session → consultation completion → notifications

1. After appointment confirmation, telemedicine session is provisioned and can be fetched via GET /api/telemedicine/sessions/appointment/{id}.
2. Doctor marks completion via POST /api/telemedicine/sessions/appointment/{id}/complete.
3. Both patient and doctor receive consultation.completed notifications (verified in smoke test).

4.4 Real-time appointment status updates (SSE)

Appointment status changes are broadcast using Server-Sent Events so the UI can update in real time.

5. Security & Authentication

- JWT is issued by auth-service and validated at the API gateway.
- The gateway enforces RBAC (role checks where configured) and injects identity headers (X-User-Id, X-User-Role, optional X-User-Email).
- Downstream services authorize requests using these headers (least-privilege per endpoint).

6. Deployment

6.1 Docker Compose

```
docker compose up --build
```

Key URLs:

- Frontend: <http://localhost:5173>
- Gateway: <http://localhost:8090>
- Eureka: <http://localhost:8761>
- RabbitMQ UI: <http://localhost:15672> (guest/guest)

6.2 Kubernetes

Manifests are under `k8s/` and include deployments/services/config/secrets/ingress plus readiness/liveness probes.

6.3 Automated proof run (smoke test)

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\smoke-test.ps1
```

The smoke test executes the complete workflow through the gateway and validates key outcomes (confirmation, session provisioning, completion notifications, admin review actions).

7. Individual Contributions

Primary ownership by member:

- LIYANAGE S.P. (IT23187450): Patient Management Service, Authentication Service (JWT issuance), Telemedicine Service, and API Gateway identity propagation (JWT/RBAC headers).
- WICKRAMATHILAKA N.M. (IT23165434): Payment Service, Doctor Management Service, payment-provider integrations (PayHere/Stripe), and admin payment review/refund flows.
- RAMANAYAKE R.H.B.D.G. (IT23164130): Appointment Service, AI Symptom Checker Service integration, real-time appointment updates (SSE), and event-driven appointment lifecycle handling.
- ALAHAKOON A.W.A.C.N. (IT23163522): Notification Service (Brevo/Twilio delivery), Prescription Service, and notification event-consumer orchestration.

Shared core integration responsibilities:

- Service discovery registration consistency (Eureka) and gateway route alignment: LIYANAGE S.P.
- Frontend-to-backend integration for payment and doctor modules: WICKRAMATHILAKA N.M.
- Cross-service appointment orchestration and workflow verification support: RAMANAYAKE R.H.B.D.G.
- Operational notification delivery reliability (email/SMS dispatch wiring): ALAHAKOON A.W.A.C.N.

8. Appendix – Code Written (NO screenshots)

8.1 JWT Token Service (LIYANAGE - Authentication)

File: auth-service/src/main/java/lk/medilink/auth/security/JwtService.java

```
package lk.medilink.auth.security;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import lk.medilink.auth.domain.UserAccount;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.List;
import java.util.UUID;

@Service
public class JwtService {
    private final SecretKey key;
    private final long ttlSeconds;

    public JwtService(@Value("${security.jwt.secret}") String secret,
                     @Value("${security.jwt.ttl-seconds:3600}") long ttlSeconds) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.ttlSeconds = ttlSeconds;
    }

    public String issue(UserAccount user) {
        Instant now = Instant.now();
        return Jwts.builder()
            .id(UUID.randomUUID().toString())
            .issuedAt(java.util.Date.from(now))
            .expiration(java.util.Date.from(now.plusSeconds(ttlSeconds)))
            .claim("uid", user.getId())
            .claim("email", user.getEmail())
            .claim("roles", List.of(user.getRole().name()))
            .signWith(key)
            .compact();
    }
}
```

```
}
}
```

8.2 Patient Profile Management (LIYANAGE - Patient Service)

File: patient-service/src/main/java/lk/medilink/patient/service/PatientAppService.java

```
@Service
public class PatientAppService {
    private static final Pattern E164 = Pattern.compile("^\\+[1-9]\\d{7,14}$");
    private static final long MAX_PHOTO_BYTES = 2L * 1024 * 1024;
    private static final Set<String> ALLOWED_PHOTO_CONTENT_TYPES =
        Set.of("image/jpeg", "image/png", "image/webp");

    @Transactional
    public PatientProfile upsertProfile(Long userId, String fullName, String phone,
                                       LocalDate dateOfBirth, String address, String
                                       String normalizedPhone = normalizeToE164(phone);
    if (normalizedPhone == null) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
            "Phone number is invalid; use E.164 format");
    }

    PatientProfile p = profiles.findById(userId)
        .orElseGet(() => profiles.save(
            new PatientProfile(userId, fullName, normalizedPhone,
            p.setFullName(fullName);
            p.setPhone(normalizedPhone);
            p.setDateOfBirth(dateOfBirth);
            p.setAddress(address);
            return profiles.save(p);
    }

    @Transactional
    public PatientProfile uploadProfilePhoto(Long userId, MultipartFile file) {
        if (cloudinary == null) {
            throw new ResponseStatusException(HttpStatus.SERVICE_UNAVAILABLE,
                "Image storage not configured");
        }
        if (file.getSize() > MAX_PHOTO_BYTES) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Photo must be 2MB or less");
        }
        // Upload to Cloudinary and store URL
        return profiles.save(p);
    }
}
```

8.3 Telemedicine Session Management (LIYANAGE - Telemedicine Service)

File: telemedicine-

service/src/main/java/lk/medilink/telemedicine/service/TelemedicineAppService.java

```
@Service
public class TelemedicineAppService {
    private final TelemedicineSessionRepository repo;
    private final RabbitTemplate rabbit;
    private final String jitsiBaseUrl;

    @Transactional
    public TelemedicineSession completeConsultation(Long appointmentId, Long requesterId) {
        TelemedicineSession s = repo.findByAppointmentId(appointmentId)
            .orElseThrow(() => new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Telemedicine session not found"

        if (s.getStatus() != SessionStatus.ACTIVE) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Session is not active");
        }

        boolean changed = s.complete();
        TelemedicineSession saved = repo.save(s);
        if (changed) {
            rabbit.convertAndSend(RabbitConfig.EXCHANGE,
                "consultation.completed",
                new ConsultationEvents.ConsultationCompleted(
                    saved.getAppointmentId(), saved.getPatientUserId(),
                    saved.getDoctorId(), saved.getSlotTime(), saved.getCom

        }
        return saved;
    }
}
```

8.4 Doctor Availability & Slot Generation (WICKRAMATHILAKA - Doctor Service)

File: doctor-service/src/main/java/lk/medilink/doctor/service/DoctorAppService.java

```
@Service
public class DoctorAppService {
    private static final int SLOT_MINUTES = 30;
    private static final ZoneId AVAILABILITY_ZONE = ZoneId.of("Asia/Colombo");

    @Transactional
    public DoctorProfile upsertProfile(Long userId, String fullName, String registrationNo,
        String specialization, Integer consultationFee) {
        DoctorProfile profile = repo.findById(userId)
            .orElseGet(() => new DoctorProfile(userId, fullName,
                registrationNo, specialization));

        profile.setRegistrationNo(registrationNo);
        profile.setSpecialization(specialization);
    }
}
```



```

        profile.setConsultationFeeLkr(consultationFeeLkr);
        if (profile.getStatus() != VerificationStatus.VERIFIED) {
            profile.resetVerificationToPending();
        }
        return repo.save(profile);
    }

    public List<TimeSlot> generateAvailableSlots(Long doctorId, LocalDate date) {
        List<DoctorAvailabilityBlock> blocks =
            availabilityRepo.findByDoctorIdAndDate(doctorId, date);
        List<TimeSlot> slots = new ArrayList<>();

        for (DoctorAvailabilityBlock block : blocks) {
            LocalTime current = block.getStartTime();
            while (current.isBefore(block.getEndTime())) {
                slots.add(new TimeSlot(current,
                    current.plusMinutes(SLOT_MINUTES)));
                current = current.plusMinutes(SLOT_MINUTES);
            }
        }
        return slots;
    }
}

```

8.5 Payment Admin Review & Refund (WICKRAMATHILAKA - Payment Service)

File: payment-service/src/main/java/lk/medilink/payment/service/PaymentAppService.java

```

@Service
public class PaymentAppService {
    @Transactional
    public Payment reviewPayment(Long paymentId, Long adminUserId, String note) {
        Payment p = repo.findById(paymentId)
            .orElseThrow(() => new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Payment not found"));
        p.markReviewed(adminUserId, note == null ? null : note.trim());
        return repo.save(p);
    }

    @Transactional
    public Payment flagPaymentDispute(Long paymentId, Long adminUserId, String note) {
        Payment p = repo.findById(paymentId)
            .orElseThrow(() => new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Payment not found"));
        p.markDisputed(adminUserId, note == null ? null : note.trim());
        return repo.save(p);
    }

    @Transactional
    public Payment refundPayment(Long paymentId, Long adminUserId, String reason) {
        Payment p = repo.findById(paymentId)
            .orElseThrow(() => new ResponseStatusException(

```

```

        HttpStatus.NOT_FOUND, "Payment not found"));
    if (p.getStatus() != PaymentStatus.COMPLETED) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
            "Only completed payments can be refunded");
    }
    p.markRefunded(adminUserId, reason);
    return repo.save(p);
}
}

```

8.6 Appointment Creation & Real-time Updates (RAMANAYAKE - Appointment Service)

File: appointment-

service/src/main/java/lk/medilink/appointment/service/AppointmentAppService.java

```

@Service
public class AppointmentAppService {
    @Transactional
    public Appointment create(Long patientId, Long doctorId, Instant slotTime) {
        if (!slotTime.isAfter(Instant.now().plus(Duration.ofMinutes(1)))) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Slot time must be in the future");
        }

        if (!doctorAllowsSlot(doctorId, slotTime)) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Doctor is not available at that time");
        }

        if (repo.existsByDoctorIdAndSlotTimeAndStatusNot(doctorId, slotTime,
            AppointmentStatus.CANCELLED)) {
            throw new ResponseStatusException(HttpStatus.CONFLICT,
                "That slot is already booked");
        }

        Appointment saved = repo.save(new Appointment(patientId, doctorId, slotTime,
            AppointmentStatus.PENDING_PAYMENT));
        rabbit.convertAndSend(RabbitConfig.EXCHANGE, "appointment.created",
            new AppointmentEvents.AppointmentCreated(saved.getId(),
                patientId, doctorId, slotTime));
        sse.publish(saved);
        return saved;
    }

    @Transactional
    public Appointment reschedule(Long appointmentId, Instant newSlotTime) {
        Appointment appt = repo.findById(appointmentId)
            .orElseThrow(() => new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Appointment not found"));
        appt.reschedule(newSlotTime);
        Appointment saved = repo.save(appt);
        sse.publish(saved);
    }
}

```

```

        return saved;
    }
}

```

8.7 Email Notifications via Brevo (ALAHAKOON - Notification Service)

File: notification-

service/src/main/java/lk/medilink/notification/delivery/BrevoEmailSender.java

```

@Component
public class BrevoEmailSender {
    private final RestTemplate rest;
    private final String apiKey;
    private final String fromEmail;
    private final String fromName;

    public boolean isEnabled() {
        return !apiKey.isBlank() && !fromEmail.isBlank();
    }

    public void send(String toEmail, String subject, String textBody) {
        if (!isEnabled()) return;
        if (toEmail == null || toEmail.isBlank()) return;

        HttpHeaders h = new HttpHeaders();
        h.setContentType(MediaType.APPLICATION_JSON);
        h.set("api-key", apiKey);

        Map<String, Object> payload = Map.of(
            "sender", Map.of("email", fromEmail, "name", fromName),
            "to", List.of(Map.of("email", toEmail)),
            "subject", subject,
            "textContent", textBody
        );

        try {
            ResponseEntity<Map<String, Object>> res = rest.exchange(
                "https://api.brevo.com/v3/smtpp/email",
                HttpMethod.POST,
                new HttpEntity<>(payload, h),
                new ParameterizedTypeReference<>() {}
            );
            if (res.getStatusCode().is2xxSuccessful()) {
                log.info("Brevo email sent to {}", safeEmail(toEmail));
            }
        } catch (RestClientException ex) {
            log.warn("Brevo email failed: {}", ex.getMessage());
        }
    }
}

```

8.8 SMS Notifications via Twilio (ALAHAKOON - Notification Service)

File: notification-

service/src/main/java/lk/medilink/notification/delivery/TwilioSmsSender.java

```
@Component
public class TwilioSmsSender {
    private static final Pattern E164 = Pattern.compile("^\\+[1-9]\\d{7,14}$");
    private final RestTemplate rest;
    private final String accountSid;
    private final String authToken;
    private final String fromNumber;

    public boolean isEnabled() {
        return !accountSid.isBlank() && !authToken.isBlank() && !fromNumber.isBlank()
    }

    public void send(String toNumber, String body) {
        if (!isEnabled()) return;
        if (toNumber == null || toNumber.isBlank()) return;

        String normalizedTo = normalizeToE164(toNumber);
        if (normalizedTo == null) {
            log.warn("Twilio SMS skipped: invalid number");
            return;
        }

        MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
        form.add("From", fromNumber);
        form.add("To", normalizedTo);
        form.add("Body", body);

        try {
            ResponseEntity<Map<String, Object>> res = rest.exchange(
                "https://api.twilio.com/2010-04-01/Accounts/" +
                accountSid + "/Messages.json",
                HttpMethod.POST,
                new HttpEntity<>(form, basicAuthHeader()),
                new ParameterizedTypeReference<>() {}
            );
            if (res.getStatusCode().is2xxSuccessful()) {
                log.info("Twilio SMS sent to {}", safePhone(normalizedTo));
            }
        } catch (RestClientException ex) {
            log.warn("Twilio SMS failed: {}", ex.getMessage());
        }
    }
}
```

8.9 Prescription Issuance (ALAHAKOON - Prescription Service)

File: prescription-

service/src/main/java/lk/medilink/prescription/service/PrescriptionAppService.java

```
@Service
public class PrescriptionAppService {
    @Transactional
    public Prescription issue(Long doctorUserId, CreatePrescriptionRequest req) {
        if (doctorUserId == null || doctorUserId <= 0) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "Missing doctor user id");
        }
        if (req.patientUserId() == null || req.patientUserId() <= 0) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "patientUserId is required");
        }

        String medications = req.medications() == null ? "" : req.medications().trim();
        if (medications.isBlank()) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "medications is required");
        }

        Prescription p = new Prescription(
            doctorUserId,
            req.patientUserId(),
            req.appointmentId(),
            blankToNull(req.diagnosis()),
            medications,
            blankToNull(req.notes())
        );
        return repo.save(p);
    }

    public List<Prescription> listForPatient(Long patientUserId) {
        return repo.findByPatientUserIdOrderByIssuedAtDesc(patientUserId);
    }

    public List<Prescription> listForDoctor(Long doctorUserId) {
        return repo.findByDoctorUserIdOrderByIssuedAtDesc(doctorUserId);
    }
}
```

End of report.